OD OpsDesk for TBO

TECHNICAL ARCHITECTURE

How OpsDesk is built

React + Vite front-end, FastAPI + SQLAlchemy back-end, scikit-learn pipeline, SQLite (Postgres-compatible) storage.

Technical Architecture Deck · 15 slides · no credentials in source

- Single deployable: the SPA is served by the same FastAPI process.
- Every number in this deck is read from a file produced by the build.
- Reference architecture and implemented MVP are kept explicitly separate.

SYSTEM

Architecture at a glance

One process serves the API and the built SPA; the browser never talks to the database.

BROWSER	React 18 · Vite · TypeScript · Tailwind · Recharts · React Router
API	FastAPI routers: auth, tickets, routing, escalations, side conversations, analytics, exports, simulation
SERVICES	extraction · classifier · linking · sla · routing · assignment · escalation · exports · analytics
DATA	SQLAlchemy 2.x models on SQLite (WAL) — 15 tables; Postgres-compatible DDL
ARTEFACTS	joblib pipeline · benchmark JSON · screenshots · decks · videos

REQUEST PATH

- Browser calls /api/* with a bearer token minted by /api/auth/login.
- FastAPI validates the token, resolves the user and enforces RBAC before the handler runs.
- Service layer runs the domain logic and writes audit rows inside one transaction.
- Pydantic serialisers shape every response; nothing is assembled in the browser.
- GET / falls back to the SPA build, with a 503 JSON hint if the build is missing.

STACK

Technology choices

Chosen for a single-node demo that still mirrors production boundaries.

Layer	Technology	Version	Why
Front-end	React + TypeScript + Vite	18.3 · 5.4	Fast dev loop, typed API contracts
Styling	Tailwind CSS	3.4	Consistent light enterprise theme
Charts	Recharts	2.13	Tooltips and filters over real payloads
API	FastAPI + Pydantic	0.162 · 2.12	Typed validation, OpenAPI, streaming
ORM	SQLAlchemy	2.1	Postgres-swappable models
Database	SQLite (WAL)	3.x	Zero-ops demo; Postgres-compatible
ML	scikit-learn	1.6	TF-IDF + logistic regression + calibration
Analytics	DuckDB (declared)	2.0	Warm-tier design; not on the request path
Tests	pytest + httpx + Playwright	—	API, journeys and real browser checks

DATA

Fifteen tables, one join key

The booking id links tickets, conversations, routing audits and analytics.

Identity	Agency, User (role, region, skill, capacity, shift)
Travel context	Booking (TBOH id, supplier, LCD, check-in, timezone, agency)
Case	Ticket, TicketLink, Message, ActivityTimeline, TicketTransfer
Conversations	SideConversation, SideMessage (explicit recipients)
Decisioning	RoutingAudit (scores, exclusions, selected agent)
Escalation	EscalationRecord (EM, reason, previous owner/status/queue)
Product	ProductBug, ThemeTag, ExportJob

INVARIANTS

- All timestamps stored as aware UTC; display converted per booking timezone.
- Audit rows are append-only and never updated in place.
- Side-conversation recipients are stored per conversation, never per group.
- Foreign keys and status enums are validated by the service layer.

MODULE 1

Extraction, classification, case linking

Two stages, one persisted artefact, a deterministic fallback.

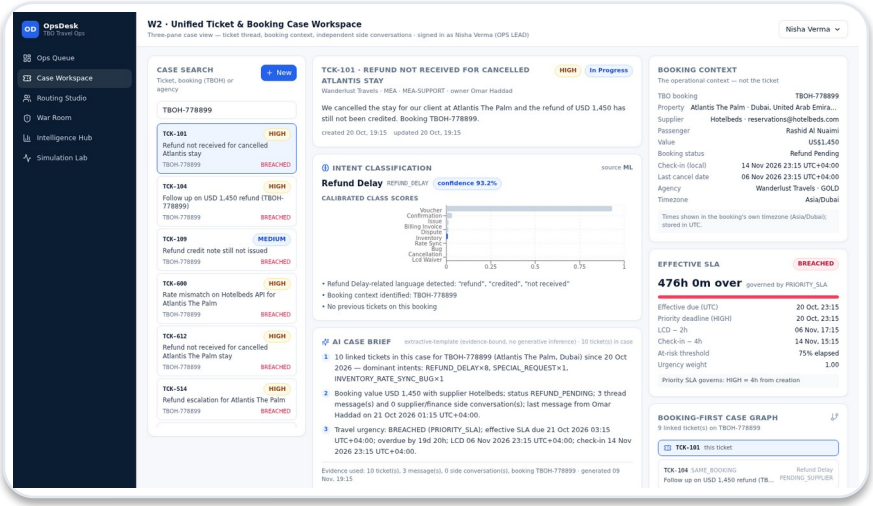
Stage A — Regex extraction of TBOH-###### plus hotel/supplier hints when the field is empty.

Stage B — TF-IDF → logistic regression → CalibratedClassifierCV; probabilities, not scores.

Explain — Reasoning lines name the signals used; the same payload feeds the UI.

Fallback — Keyword rules if the joblib file is missing — the API reports mode RULES.

Link — ticket_links connect every ticket sharing the booking, with a link reason.



Classification card in the running app.

Holdout: 0.8958 accuracy, 0.8964 macro F1 (n=288, template-disjoint).

MODULE 2

Routing engine internals

Filters, score, tie-break, persist — in that order.

```
35*RegionMatch + 25*ShiftMatch + 25*(SkillRank/5)*UrgencyWeight + 15*(1 - ActiveLoad/MaxCapacity)
```

Step	Rule	Result
1 · Candidates	OPS_AGENT role only; escalation managers and the lead are never scored	Pool
2 · Hard filters	offline · at capacity · outside shift · skill < 4 for CRITICAL/AT_RISK/BREACHED · regional eligibility	Excluded list
3 · Score	region 35 · shift 25 · skill 25 × urgency · capacity 15	0–100
4 · Tie-break	highest score → lowest active load → lowest agent id	Winner
5 · Persist	RoutingAudit row with every candidate, every component and the reason	Audit

Worked example (live): TCK-201, APAC, BREACHED → urgency 1.0. Ananya Sharma scores 98.13 = 35 region + 25 shift + 25 skill + 13.13 capacity, with 3 eligible and 6 excluded candidates. The comparator is a pure function of stored state, so the same inputs always reproduce the same decision.

MODULE 3

Conversations and transfer safety

The main thread and side conversations are different objects with different rules.

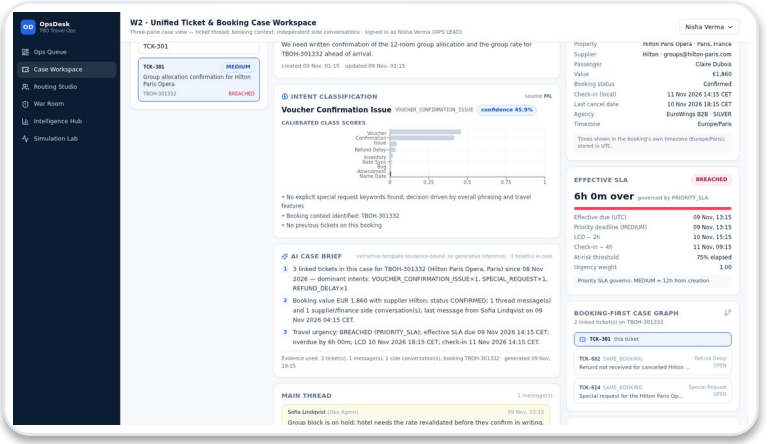
MAIN THREAD

- Public replies visible to the agency requesters.
- Internal notes stored on the ticket but excluded from the agency view.

SIDE CONVERSATIONS

- Supplier / finance / agency threads with explicit recipient lists.
- Own message history, own status, independent of the main thread.

Transfer step	Effect on conversations
Queue change	Ticket moves, timeline event written
Owner change	Previous owner recorded on the transfer row
Side conversations	Conversation ids, recipients and history preserved
Destination group	Addresses never inherited by existing conversations
Verification	Recipients compared before/after and returned as recipients_unchanged



Supplier thread with explicit recipients.

MODULE 4

SLA engine

Computed on read from stored dates; never a static status flag.

EffectiveSLADueAt = MIN(PriorityDeadline, LCD − 2h, CheckIn − 4h)

- Priority windows: CRITICAL 1h · HIGH 4h · MEDIUM 12h · LOW 24h.
- Governing rule is returned with each ticket so the UI can explain the clock.
- AT_RISK at 75% elapsed; 50% inside a regional holiday, with the reason recorded.
- Missing dates are listed; already-expired travel deadlines are ignored explicitly.
- Urgency weight from the SLA state feeds the routing score.

State	Meaning
HEALTHY	Before the at-risk threshold
AT_RISK	Past the threshold, before the due date
BREACHED	Due date passed (countdown shows time over)
RESOLVED	Closed before or after the due date

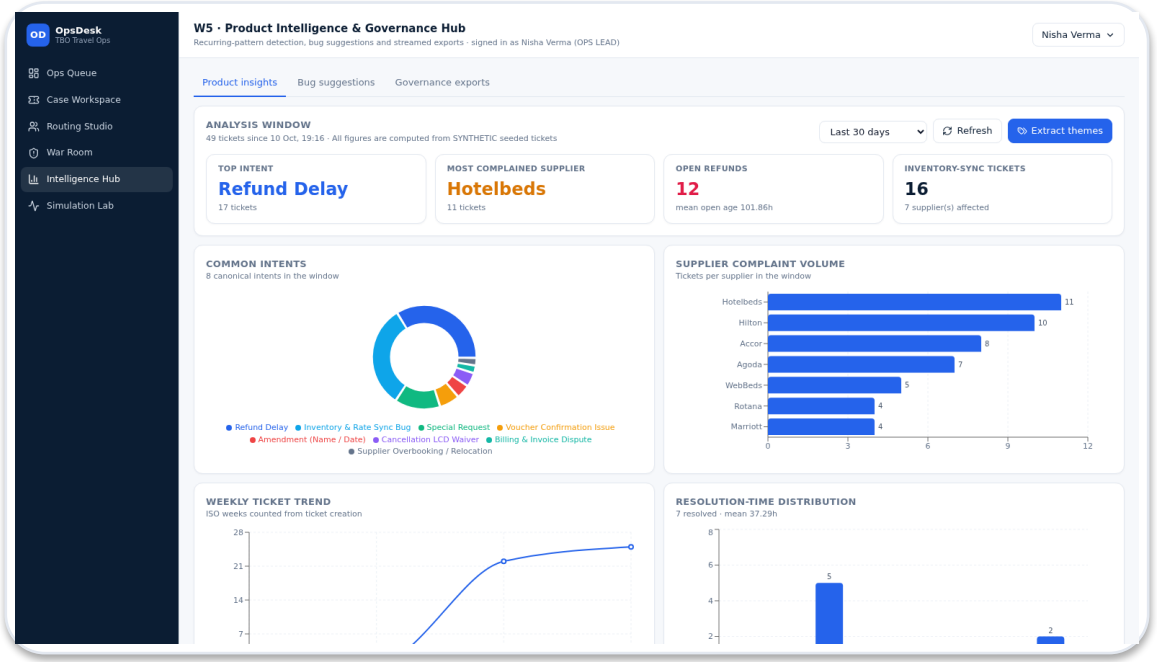
Escalation: POST /api/escalations/tickets/{id}/escalate writes the reason, picks the eligible manager for the region, moves status to ESCALATED, queue to ESCALATION_MANAGER, reassigns ownership and appends an immutable event — atomically.

MODULE 5

Analytics and pattern detection

Aggregations over stored rows, plus a deterministic pattern rule.

- Intent distribution, supplier complaint volume, weekly trend, resolution-time histogram.
- Refund bottleneck: open count, mean open age, oldest open ticket.
- Regional performance: volume, breaches, resolved, mean resolution hours.
- Inventory/rate-sync cluster: ticket count, suppliers, ticket numbers.
- Theme extraction tags recurring language, persisted per ticket.
- Bug proposals: group by (supplier, intent), require ≥ 3, attach evidence and period.



Product intelligence tab, windowed to 30 days.

Records land in ProductBug inside this database. No external tracker is contacted — the limitation is stated in the UI.

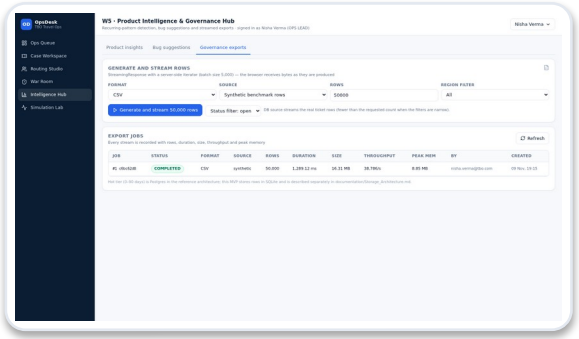
MODULE 6

Streaming export internals

Memory stays flat because rows are yielded, never collected.

- POST /api/exports/stream returns a StreamingResponse over a generator.
- Rows are produced in batches of 5,000; filters are applied in SQL.
- Job row is created first, so progress and errors are observable.
- Duration, bytes, throughput and VmRSS are measured around the stream.
- GET /{uuid}/validate re-reads the file and compares row + column counts.
- GET /{uuid}/download serves the written file with the bearer token.

Measured	Value
Rows	50,000
Columns	25
File size	16.313 MB
Server duration	1,289.12 ms
Throughput	38,786 rows/s
Peak memory	8.848 MB



Export tab before streaming.

Storage tiering (hot/warm/cold) is documented separately as a reference architecture — the MVP is a single SQLite file.

SECURITY

Access control and data handling

RBAC is enforced in the API; the UI only mirrors it.

Capability	Agency requester	Ops agent	Escalation manager	Ops lead
Ticket visibility	own agency	region	region	all
Create ticket	yes	yes	yes	yes
Main thread reply	yes	yes	yes	yes
Internal note	no	yes	yes	yes
Route / reassign	no	yes	yes	yes + override
Side conversations	yes	yes	yes	yes
Escalate	no	yes	yes	yes
Product intelligence	no	no	yes	yes
Streamed exports	no	no	yes	yes
Run simulation	no	no	yes	yes
Reseed / reset dataset	no	no	no	yes

CONTROLS

- PBKDF2-SHA256 password hashing, per-user salt.
- Signed bearer tokens with expiry; role and agency in the claims.
- Pydantic validation on every request body and query.
- Environment-driven config via .env; no secrets in source.
- Immutable audit rows for routing, escalation and transfer.

QUALITY

Testing strategy

Unit, module, system and browser level — the browser level runs against the built app.

Layer	What it proves	Count / result
pytest module tests	SLA, routing (incl. 7 mandatory cases), conversations, exports, analytics, RBAC	104 tests
Backend smoke	OpenAPI surface and health contract	2 tests
Seeded journeys	A–E end-to-end against the live API	5/5 · 34/34 steps
Export validation	Row count, headers, readability, duration, memory	PASS
Browser crawl	Six workspaces, console + HTTP error capture, downloads	0 errors

Unexecuted checks are reported as unexecuted in documentation/Testing_Report.md — nothing is marked passed without a run. Screenshots and JSON reports from the crawl are kept under screenshots/ and /tmp/ui_report.json for the session.

STORAGE

Reference tiering vs implemented MVP

Design intent is documented; the demo is honest about what runs today.

Hot 0–90 days	PostgreSQL, row-per-ticket, indexed on (region, status, due_at)	Reference
Warm 90 days – 2 years	DuckDB / ClickHouse columnar store for analytics	Reference
Cold 2–7 years	Parquet on S3 / MinIO, queried through DuckDB	Reference
This MVP	Single SQLite database in WAL mode on the app node (49 tickets, 12 bookings)	Implemented

DuckDB is declared as an optional analytics dependency; it is not on the request path of this build.

OPERATIONS

Run, seed and reset

Three commands from a clean checkout; the same build serves API and UI.

1 · Install	`pip install -r requirements.txt · cd frontend && npm install`
2 · Build UI	`npm run build → backend/static (served by FastAPI)`
3 · Seed	`python -m database.seed_data.seed (49 tickets, 12 bookings, 16 users)`
4 · Run	`uvicorn backend.main:app --host 0.0.0.0 --port 8000`
5 · Verify	`pytest tests backend/tests · python -m simulation.run_all`
6 · Observe	`GET /api/health reports counts, classifier mode and dataset notice`

CONFIG

- OPSDESK_SECRET_KEY
- OPSDESK_DATABASE_URL
- OPSDESK_EXPORT_DIR
- OPSDESK_ENABLE_ROLE_SWITCH
- OPSDESK_DEMO_PASSWORD

A booking is the operational context. A ticket is only one interaction associated with that booking.

— Booking-first model, audited routing, travel-aware SLA

— Measured: 106 tests, 5/5 journeys, 0 browser console errors

— Reference architecture and MVP kept honestly separate